

Lab 4 – Processor Design

IS1200/IS1500 staff

2026

Contents

1	Introduction	2
1.1	Preparations	2
1.2	Software	2
1.3	Resources and Reading Guidelines	3
2	Examination	3
3	Assignments	3
3.1	Assignment 1: Arithmetic/Logical Unit (ALU)	4
3.2	Assignment 2: Register File	6
3.3	Assignment 3: Control Unit	6
3.4	Assignment 4: Data Path	7
3.5	Assignment 5: Factorial Function in Assembler	10
4	Version History	12

1 Introduction

Welcome to this lab! In this laboratory exercise, you will learn the fundamentals of processor design. After finishing this lab, you should be able to

1. Construct a simple single-cycle processor that executes an actual assembly program.
2. Explain how bit selection and instruction decoding work.
3. Explain the basic principles of the data path and the control unit.

1.1 Preparations

You must do all assignments in advance. The teachers will examine your skills and knowledge at the lab session.

Your group submits its circuit to the Canvas assignment **Lab 4 Files + Upload code at least one hour before your booked exam slot**. The auto-grader runs hourly and reports results in your Canvas grade comments. You may re-submit as often as you like up to that point, but each attempt is logged.

We recommend that you book a lab session well in advance, and start preparing at least a week before the session.

You can ask for help any time before the lab session. There are several ways to get help; see the course website for details and alternatives. We especially encourage you to post questions on the KTH Canvas page.

1.2 Software

In this lab, we use the open-source software **Logisim Evolution v4.1.0**. It is **very important** that you download *this exact version* – the same version the auto-grader runs and the same version in which the template circuit file `processor_riscv.circ` was saved. Other versions (older or newer) may render the circuit incorrectly, refuse to open the file, or produce subtly different simulation results that fail the auto-grader.

On the KTH lab computers, Logisim Evolution v4.1.0 is already installed – you can start it directly there, with no download or setup. The instructions below are only needed if you want to work on your own computer.

Download the file `logisim-evolution-4.1.0-all.jar` from this exact release URL:

github.com/logisim-evolution/logisim-evolution/releases/download/v4.1.0/logisim-evolution-4.1.0-all.jar

(the file is approximately 50 MB). Run it with:

```
java -jar logisim-evolution-4.1.0-all.jar
```

You need Java 11 or later installed.

If you have not performed the LD-Lab, you must learn the basics of using this software before you do Lab 4.

To teach yourself how to use the Logisim Evolution tool, please go through Logisim’s “Beginner’s tutorial.” This tutorial can be found in the documentation section on Logisim Evolution’s web pages: github.com/logisim-evolution/logisim-evolution/wiki. Note that you need to perform each of the steps using the software to really learn how to use the tool.

Note: Logisim Evolution has some bugs. If you notice something that makes no sense (for example a value that does not update, or a gate giving an impossible result), closing and reopening the file typically solves it.

If you are taking course IS1200, you should already know the basics of digital design and the content of the LD-Lab (Logic Design), that is only examined for students taking IS1500. However, if you find Lab 4 a bit hard, we recommend that you perform at least parts of the LD-Lab to refresh your knowledge about digital design.

1.3 Resources and Reading Guidelines

Review the following course material while you are preparing your solutions.

- Lectures 9.
- Harris & Harris (2021) course book. Chapters 5.2.4 (about the ALU), and 7.1, 7.3 (about the single-cycle processor). The book is available online.
- The RISC-V Reference Sheet.
- The RISC-V Instruction Set Manual (Volume I: Unprivileged Specification).
- See the course web page *Literature and Resources* for links and details.

2 Examination

Lab 4 is examined in two phases:

- **Homework.** Your group submits one `processor_riscv.circ` file to the Canvas assignment **Lab 4 Files + Upload code** (one submission per group). The auto-grader runs hourly and posts results in your Canvas grade comments.
- **On-site session.** Each student is called individually to present and answer questions about *their group's submitted solution* for a randomly chosen part of the lab. You may be asked to walk through any of your sub-circuits, justify any control-signal decision, or explain any part of your factorial program. The teacher in the session decides whether your answers demonstrate a satisfactory understanding of the group's solution. The auto-grader's homework pass/fail is a prerequisite for attending the on-site session: **all five assignments (1–5) must show pass at least one hour before your booked session.** You may submit partial work earlier and re-submit as you finish each part; the auto-grader reports pass/fail per assignment after each hourly run. The on-stage oral is the final pass/fail decision.

See the course Canvas page for booking rules and deadlines. The examiner reads a per-student briefing of your auto-grader status before you arrive, so there is no re-running your circuit during the session.

3 Assignments

Throughout all assignments, test your components carefully by trying different input data and checking the output against expected output.

Note: Before you start, download file `processor_riscv.circ` from the course website. This is the template project in which you should implement *all* your solutions. Open this file in Logisim Evolution and start your processor design journey!

3.1 Assignment 1: Arithmetic/Logical Unit (ALU)

The ALU is the heart of the processor and is the digital building block that performs the actual computations. The purpose of this assignment is to gain a deeper understanding of how an ALU can be designed by implementing a basic ALU. The ALU is very similar to the one presented at Lecture 9, with some minor differences/simplifications.

Task 1.1 Open circuit **ALU** in the `processor_riscv.circ` project. Implement the ALU with the following properties:

- One 32-bit wide data input port named **A**.
- One 32-bit wide data input port named **B**.
- One 3-bit input port named **F** that determines the function of the ALU.
- One 32-bit data output port named **Y** that outputs the result of the selected function.
- One 1-bit output port named **Zero**. For the arithmetic operations **ADD** ($F=000$) and **SUB** ($F=001$), **Zero** must output value 1 if all bits of **Y** are zero and value 0 otherwise. (The control unit consumes **Zero** during **beq**, which uses **SUB**.) For the logical operations **AND**, **OR**, and **SLT** the Zero output is not checked and may be left as *don't care*.

Note that the input and output components and some other helper components are already located in the **ALU** circuit.

The ALU should implement the following functionality:

- $F = 000_2$ should perform operation $A + B$.
- $F = 001_2$ should perform operation $A - B$.
- $F = 010_2$ should perform operation $A \text{ AND } B$.
- $F = 011_2$ should perform operation $A \text{ OR } B$.
- $F = 101_2$ should perform operation *set less than (SLT)*: if $A < B$ then the ALU should output value 1, else it should output value 0 (you may use the *comparator* component).

For all other values of F , the output is undefined (we do not care what the output is).

The layout will look like this when another circuit uses the ALU circuit:

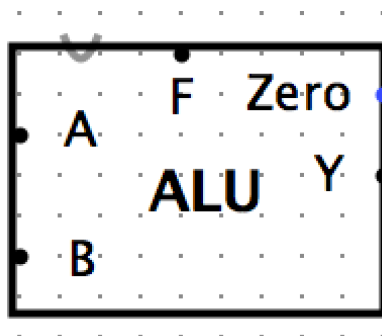


Figure 1: ALU component layout as it appears when instantiated in another circuit.

The layout for the component is already provided in the project.

Note: You are not allowed to use the Subtractor component in Logisim Evolution for this lab. However, you are allowed to use all the other pre-defined components.

Some advice:

- Input pins and output pins can be configured to be 32-bit. Note, however, that the most significant byte is displayed at the top and the least significant byte at the bottom. Take a look at Example 1 in circuit **Tests** (part of the project) and convince yourself of the bit structure.
- The ALU is very similar to the one in Harris & Harris in Section 5.2.4 and Lecture 9. Note however the new **Zero** output port. To create an output port, add a *Pin* (from the *Wiring* library) and set its *Output?* attribute to *Yes*; then drive it with logic of your choice. Because the ALU uses a custom symbol, the port of a newly added pin is *not* placed automatically in a sensible spot on the box. After adding the **Zero** pin, open **Project** → “Edit Circuit Appearance”, drag the **Zero** port to a clear position next to its label, then return with **Project** → “Edit Circuit Layout”.
- One option to perform the zero extension (for the SLT operation) is to use a single *Splitter* component to extract the sign bit, followed by a *Bit Extender* component, which can be found in the *Wiring* library folder. As an alternative, use the **Bit Selector** component that can be found in the **Plexers** library folder.
- **Constants vs. inputs:** whenever you need a *fixed* value inside a circuit — for example a constant to compare the ALU result against, a 0 for a carry-in, or the value 4 used later to increment the PC — use a *Constant* component (from the *Wiring* library), *not* an input Pin. An input Pin creates an external port that must be driven from outside the sub-circuit; if you leave it unconnected it floats (undefined) and your logic misbehaves.

What the auto-grader checks The auto-grader runs your **ALU** sub-circuit against two sets of test vectors:

- *Output Y across all functions.* Nine input combinations covering ADD (including a case that overflows to zero), SUB (including a case that underflows to a negative number), AND, OR, and SLT in both directions ($A < B$ and $A \geq B$). For each, the auto-grader compares the 32-bit *Y* your ALU produces against the expected value.
- *Zero output for ADD and SUB.* Six input combinations where the auto-grader additionally checks that **Zero** = 1 exactly when $Y = 0$ (for example, $5 + (-5) = 0$ should set Zero, and $5 - 3 = 2$ should clear Zero). Zero is not checked for AND/OR/SLT – per the spec above, it is don’t-care for those functions.

A passing run means your ALU implements all five operations correctly and signals Zero correctly for the arithmetic operations the control unit consumes. Each operation is checked separately, so if one fails the feedback names it (e.g. SLT) together with the inputs, and you know exactly which function to fix.

What the auto-grader expects Sub-circuit ALU. Keep pin labels and widths exactly: inputs A, B (32-bit), F (3-bit); output Y (32-bit). You also add a 1-bit output **Zero**. Pin positions are free.

Questions for Assignment 1 These questions are useful for preparing for the oral examination. However, the teacher may choose to ask questions that are not on this list at the examination.

- Explain how each of the ALU functions are defined. In particular, you need to be able to explain how subtraction works, including the use of two’s complement.

- How did you implement the logic for the **Zero** output port? Did you consider any alternatives? Be prepared to explain your design choices.
- What is the purpose of the ALU? Why are several functions grouped together into one component?

3.2 Assignment 2: Register File

In this assignment, you will implement a register file with 2 read ports and 1 write port. The register file will then later be used to store the register values for the processor. This register file will only be able to handle the first eight registers (0, ra, sp, gp, tp, t0, t1, and t2). All registers are 32-bit. While this simplification forces us to use a rather limited (and non-standard) way of using the registers, we do this to avoid making the circuits too complex.

In circuit **RegisterFile**, an almost complete 3-bit addressed register file is provided.

Task 2.1 Register 0 (also called **zero**) has a special semantics. Implement the correct behavior for register 0 in the circuit.

Task 2.2 For debugging purposes in later assignments, it is convenient to output the value for register **t2**. Add connections to the circuit so that register **t2** is sent to output port **t2**.

What the auto-grader checks The auto-grader writes a distinct value to every writable register (**ra..t2**) and reads them all back through both read ports, checking that write works on each register and that writing one register does not disturb the others; a separate check confirms that register 0 (**zero**) stays zero even when written. If a read is wrong, the feedback names the specific register and the value that was written to it.

What the auto-grader expects Sub-circuit **RegisterFile**. Keep pin labels and widths exactly: inputs CLK, WE3 (1-bit), A1, A2, A3 (3-bit), WD3 (32-bit); outputs RD1, RD2, T2 (32-bit). Pin positions are free.

Questions for Assignment 2 These questions are useful for preparing for the oral examination. However, the teacher may choose to ask questions that are not on this list at the examination.

- Explain if the read, write, or both operations are clocked (updated at the clock edge). Why is it implemented this way?
- Explain the semantics of reading from and writing to register 0, and how you implemented this behavior.
- How many bits of data can this register file store? If the address width were the same size as that of a complete 32-bit RISC-V processor, how many bits would the register file store?

3.3 Assignment 3: Control Unit

In this assignment you will implement a small control unit. You need to have a good understanding of both the control unit and the data path for the single-cycle processor, as presented at the lectures and in the course book. Note also that this control unit is a simplified version of the unit presented at the lectures and in the book; it cannot handle all instructions.

Circuit **ControlUnit** is prepared with a correct (but not optimal) implementation that can handle instructions **addi** and **add**.

Hint: the prepared implementation uses *Comparator* components that match parts of the instruction against constants. To understand what those constants are, and to work out the values you need for **beq**, consult the *RISC-V reference sheet* on the course *Literature and Resources* page.

Task 3.1 Add logic and components to circuit **ControlUnit** so that it can also handle the correct control signals for instruction **beq** (*hint:* use the ALU to test for the branching condition). Note that the **ALUControl** and **Branch** outputs are currently driven by fixed values that are correct for **addi** and **add** but not for **beq**; you will need to change this wiring so that these signals take the right values for **beq** as well.

What the auto-grader checks A handful of instruction encodings covering **addi**, **add**, and **beq** are fed to **Instruction**; for each, the four control outputs (**RegWrite**, **ALUSrc**, **ALUControl**, **Branch**) are compared against the expected values. The unit is purely combinational; no clock involved. The provided instructions (**addi**, **add**) and the **beq** you add are checked as separate groups, so breaking **addi**/**add** while wiring **beq** is reported on its own; the feedback lists all four control signals for each failing instruction.

What the auto-grader expects Sub-circuit **ControlUnit**. Keep pin labels and widths exactly: input **Instruction** (32-bit); outputs **RegWrite**, **ALUSrc**, **Branch** (1-bit), and **ALUControl** (3-bit). Pin positions are free.

Questions for Assignment 3 These questions are useful to prepare yourself for the oral examination. However, at the examination, the teacher may choose to ask questions that are not on this list.

- Explain how you have implemented the control signals for the **beq** instruction. Why is this a correct solution?

3.4 Assignment 4: Data Path

In this task, you should create a data path that can handle three RISC-V instructions correctly: **addi**, **add**, and **beq**. This is the most demanding assignment in this lab, so follow the instructions below carefully. It is very important that you verify each of the instructions using the supplied test program.

Reminder: as noted in Section 1.2, Logisim Evolution has some bugs. If a value looks non-sensical and you are convinced the wiring is right, close and reopen the file before hunting for a bug that is not there.

Task 4.1: Updating the program counter and fetching instructions

1. Go to the **Datapath** circuit. All components in this assignment should be added to this circuit.
2. The circuit already contains the register for the PC counter and the connection to the instruction memory. Add logic for incrementing the PC with value 4 (next instruction). Note that the address width of the program counter is 8 bits, so that we do not need to allocate a too large ROM memory. Test the circuit with the provided clock.
3. The code memory should be implemented using the read-only memory (ROM) component in Logisim Evolution (under library directory *Memory*). This component is already provided in the **Datapath** circuit. The address bit width is 8 bits and the data bit width is 32 bits. The ROM is *word-addressed*: address 0 selects the first 32-bit word, address 1 the

second word, and so on. The PC, however, holds a *byte* address and advances by 4 per instruction (0, 4, 8, ...). To convert the byte address into the corresponding word index, we have added a shift component before the ROM address input (shift right by 2, i.e. divide by 4): so byte address 0 maps to word 0, byte address 4 to word 1, byte address 8 to word 2, and so on. Initiate it and test that you can fetch instructions one after another. To see that it works correctly, it is good to add an output port for observing output during testing.

Task 4.2: Implementing the `addi` instruction

1. The first thing to do is to create a test program that we can use to verify that the operation works. Download the program *test.S* from the course web page. Open the program in the RARS simulator and go to the execute view (shown below). In the **Code** column, we have the machine code for the instructions. Run the program and convince yourself of its behavior.

Address	Code	Basic
0x00400000	0x00400293	<code>addi x5,x0,4</code>
0x00400004	0x00228393	<code>addi x7,x5,2</code>
0x00400008	0x005383b3	<code>add x7,x7,x5</code>
0x0040000c	0x00538663	<code>beq x7,x5,0x0000000c</code>
0x00400010	0xffd38393	<code>addi x7,x7,0xffffffffd</code>
0x00400014	0xfe000ce3	<code>beq x0,x0,0xffffffff8</code>
0x00400018	0x00000033	<code>add x0,x0,x0</code>

Figure 2: Execute view in RARS showing machine code (Code column) for the test program.

2. Enter the machine code for the two first instructions in the ROM memory (the two first **`addi`** instructions). Right-click on the ROM and select “Edit contents”. Write the encoding of the instructions into the cells; the edit view should then look like Figure 3.

Hex Editor					
00	00400293	00228393	00000000	00000000	00C
08	00000000	00000000	00000000	00000000	00C
10	00000000	00000000	00000000	00000000	00C
...	...	...	...	...	...

Figure 3: ROM edit view after entering the first two **`addi`** instructions.

When you are done, click “Close Window”.

NOTE: you might need to save the content to a file and then load it again to be able to update the memory in the component.

3. Add the **RegisterFile** and the **ALU** components. See the course book and Lecture 9 for details about how you implement an immediate instruction. Since this task handles only **`addi`**, the second ALU operand is always the immediate, so you can connect the immediate directly to the ALU. You do *not* need the **ALUSrc** multiplexer yet; you will add it in Task 4.3, where the **`add`** instruction makes the second operand come from a register. Note that the address width is 3 bits for the **RegisterFile**, which means that you should only use the 3 least significant bits of the RISC-V addresses after decoding. To select bits, use the **Splitter** (or, if you prefer, the **Bit Selector**) component (found in the *Plexer*

library folder). In the circuit **Tests**, there are two examples that show how the Splitter component can be used. You probably need to play with these examples to understand how it works.

4. Attach the data path to the relevant control signals of the control unit. Do not forget to feed the instruction into the control unit.
5. Test that the execution sequence of the 2 instructions works as expected. You may need to add a few output components to see what happens in the circuit.

Task 4.3: Implementing the add instruction (R-type)

1. Open file *test.S* again in RARS and enter the three first instructions to the ROM component (including the third instruction, which is an **add** instruction).
2. Implement the necessary logic for the **add** instruction in the data path. Note that you need to add one or more multiplexors — in particular the **ALUSrc** multiplexer, which now selects the second ALU operand between the immediate (for **addi**) and the register value (for **add**). Connect these multiplexor(s) to the right control signals.
3. Test that the sequence of two **addi** and one **add** instruction works correctly.

Task 4.4: Implementing the beq instruction

1. Add the logic for the **beq** instruction. Connect the relevant control signals.
2. Implement the rest of the test program *test.S* into the ROM. Initialize the PC to 0x14 and check that the jump is performed.

What the auto-grader checks The auto-grader loads its own small programs into the Datapath's ROM and drives the clock, sampling the **t2** output. It checks each instruction on its own — **addi**, **add**, and a taken **beq** — with a one- or two-line program each, and then runs the full *test.S* program as an integration test (verifying **t2**'s progression, including both branch-taken and branch-not-taken **beq**). Because the grader loads these programs itself, you do *not* need to leave *test.S* in the ROM for this assignment.

What the auto-grader expects Sub-circuit **Datapath**. Required output pin: **t2** (32-bit). The PC must reset to 0x0 when the simulation starts. You may add other internal/debug pins; they are ignored.

Questions for Assignment 4 These questions are useful to prepare yourself for the oral examination. However, at the examination, the teacher may choose to ask questions that are not on this list.

- Explain how the bit selection works. Which instructions are using what logic and why?
- Explain how the **beq** instruction is implemented, how the address is calculated, and how the control unit controls the signals.
- What is the purpose and usage of the *ImmSrc* and *ALUSrc* control signals (using Figure 4)?
- Unlike the control signals in the slides and in the literature, which include an *ImmSrc* signal, our processor gets away without one. Convince yourselves why (*hint*: what is the maximal size our PC can address, and what implications does this have on reading the 12-bit immediate offset from branch instructions?).

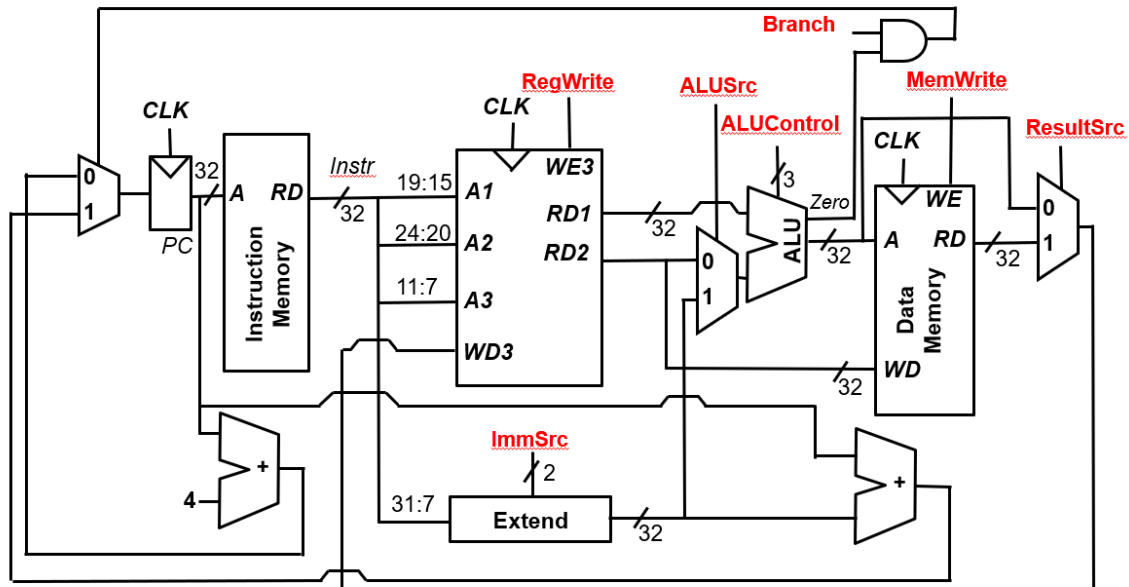


Figure 4: Single-cycle processor data path showing the ImmSrc and ALUSrc control signals, among others.

3.5 Assignment 5: Factorial Function in Assembler

In this task, you should develop a small RISC-V assembly program and execute it on your new processor.

Task 5.1 Open the RARS simulator and implement an assembly program that computes the factorial number $n!$, where n is an input parameter. The program has the following requirements:

- Only three instructions are allowed to be used: **add**, **addi**, and **beq**.
- Only registers with numbers 0 to 7 are allowed to be used. That is, you are allowed to use registers *t0*, *ra*, *sp*, *gp*, *tp*, *t0*, *t1*, and *t2*.
- The input value should be given in *t0*. That is, the first instruction should assign *t0* to value n .
- The final result should be stored in *t2*.
- At the end of the program, implement a “stop” loop, i.e., a loop that jumps to itself forever.

Note that you do not have a **mul** operation available. Hence, you should implement the multiplication by performing a loop with **add** instructions. This is not the fastest implementation, but it is good enough for testing your processor!

Task 5.2 In the RARS simulator, test that the program works for at least $0!$, $3!$, and $8!$.

Task 5.3 Move the machine code of the complete program into your processor’s ROM memory. Test that your processor gives the correct result for at least $0!$, $3!$, and $8!$.

Some advice: You can easily move your machine code from the RARS simulator into the ROM memory in Logisim Evolution.

1. In RARS, assemble the program.
2. Choose “Dump Memory...” from the File menu in RARS.
3. In the “Dump Memory To File” dialog window, set “Dump Format” to “Hexadecimal Text”.
4. Click “Dump To File...” and save the file on your system as a `.txt` file, for example `riscv1.txt`.
5. Use a plain text editor, such as *Notepad* or *Gedit*, to open the file (`riscv1.txt`), and add a new line at the very beginning of the file with the following contents:

```
v2.0 raw
```

6. Save the file, then load it into Logisim Evolution (“Load Image”).

What the auto-grader checks Two independent sub-tests:

- *5.A (assembly logic)*. `factorial.S` is assembled in RARS and `t2` is checked against `n!` for several values of `n`. Tests your algorithm in isolation.
- *5.B (end-to-end)*. The same property is checked by running your Datapath on the program loaded in your ROM, reading `t2` after enough clock cycles.

If 5.A passes and 5.B fails, the issue is in your processor; if both fail, the issue is likely in your `factorial.S`.

What the auto-grader expects Submit **two** files: `processor_riscv.circ` (factorial loaded in ROM) and `factorial.S` (the source). To let the grader vary `n` per test without parsing your code:

- In `factorial.S`, the first instruction line must be `addi t0, zero, n`, and the last instruction line must be the stop loop.
- In the Datapath’s ROM, the first 32-bit word must be the matching encoding of that `addi`. This is automatic if you followed Task 5.3 to dump from RARS.

All Assignment 1–4 contracts still apply; the result is read from the Datapath’s `t2` output pin.

Questions for Assignment 5 These questions are useful to prepare yourself for the oral examination. However, at the examination, the teacher may choose to ask questions that are not on this list.

- Show and explain how the factorial function works for arbitrary input value `n` (the teaching assistant will give you the value that you should test). Be prepared so that you know how to change the input value easily.
- Explain how you implemented unconditional jumps in your program.

Optional: Another way of compiling and transferring your code As an alternative to dumping the memory from the RARS simulator, you can use the RISC-V compilers to perform the compilation and extraction, although it is somewhat more complex. To do this:

1. Download the *time4riscv.zip* file from the Canvas homepage and extract it into a directory (this is the same file used in the labs for the board).
2. Remove all files from the directory except *boot.S*, *Makefile*, and *dtekv-script.lds*.
3. Edit *boot.S* and remove all its contents (make it empty). The reason is that your processor only supports the **addi**, **add**, and **beq** instructions, whereas the *boot.S* code uses other instructions.
4. Create a new file containing the factorial code, and compile everything by running: **make**
5. If successful, you will have a *main.bin* file containing your machine code. Next, dump the hex content, for example:

```
hexdump -ve '1/4 "%.4x "' main.bin
```

6. Copy and paste the dumped content into the ROM in your Logisim Evolution processor.

4 Version History

1.0 – First published version. 2024-07-30.

1.5 – Removed the individual-variant assignment and appendix for now. Added a “What the auto-grader checks / expects” callout to each of Assignments 1–5. 2026.

1.4 – Clarified that the ALU’s **Zero** output need only be meaningful for the operations consumed by the control unit (F=000 ADD, F=001 SUB); “don’t care” for the other functions. Logisim Evolution v4.1.0 pinned explicitly with direct download URL. 2026.

1.1 – Two-phase examination (homework on Canvas + on-site session), Logisim Evolution mandated, single *processor_riscv.circ* template. 2026.